STAR Project

Comprehensive Gap Analysis & Implementation Roadmap

Organized by Codebase Directory

Date	2026-02-18
Branch	`feat/gap-analysis-and-planning`
Overall Completion	~75%
Open PRs	9 active (see annotations throughout)
Next Milestone	First robot motion (SPI bridge + UI real data)

Confirmed complete (corrected from initial analysis):
WireMessage dispatcher: **DONE** (`comm_task.c:1959` – SetVelocity, E-Stop, PID, RetransmitConfig)
BMS telemetry streaming: **DONE** (`telemetry_task.c:1833` – voltage_v, soc_percent)

Notation: Items marked **PR #NNN** have an active pull request.
Scope: This is a prototype. OTA firmware updates are a future enhancement, not a current requirement.

STAR (Simultaneous Tracking and Robotics) – Locked Inc.

Contents

Executive Summary

STAR is a distributed robotics platform with a custom PCB RX72N motor controller, Raspberry Pi 5 control system, ROS2 Jazzy middleware, and a Go gateway service. The system communicates over 10 Mbps SPI using Protocol Buffers with HARQ/FEC error correction.

System Architecture

Directory	Language	Role
`e2-studio-star-rx72n-firmware/`	C23, GNURX, ThreadX	Motor controller firmware
`star-gateway/`	Go	UI/ROS2 bridge, SPI driver, frame protocol
`star-ros2/`	C++17, ROS2 Jazzy	Robot middleware, SPI bridge, safety monitor
`star-ui/`	TypeScript, React 19	Operator interface
`star-proto/`	Protocol Buffers 3	Message schemas (10 files, 72 messages)
`star-rpi5-buildroot/`	Buildroot	Custom Linux image for RPi5
`matlab/`	MATLAB	Motor system ID and PID design
`schematic/`	KiCad	Custom PCB designs (MCU, Power, BMS)

Communication Flow

UI
WebSocket / REST

→

Gateway (Go)
gRPC server on RPi5

→

ROS2 (C++)
`/cmd_vel`, `/robot_status`

↓

SPI bridge
10 Mbps SPI

↓

RX72N
C23 + ThreadX

Overall Completion Matrix

Component	Complete	Status	Note / PR
e2-studio-star-rx72n-firmware/			
Firmware core (9 ThreadX tasks)	100%	**DONE**	
Comm stack (HARQ/FEC/CRC-32)	100%	**DONE**	
WireMessage command dispatcher	100%	**DONE**	

Component	Complete	Status	Note / PR
BMS telemetry streaming	100%	**DONE**	Fix: **PR #342**
Telemetry USB/SPI failover	95%	**PR #342**	PR open
ThreadX stack checking	95%	**PR #343**	PR open
BMS SoC thresholds	95%	**PR #344**	PR open
Event-flag cleanup	95%	**PR #345**	PR open
Frame decoder resync	95%	**PR #341**	PR open
ASCII source cleanup	95%	**PR #346**	PR open
NVS configuration persistence	0%	**MISSING**	Not started
OTA firmware update handler	0%	**MISSING**	Future enhancement
star-gateway/			
Transport layer (SPI/USB CDC)	100%	**DONE**	
Frame codec (HARQ/FEC/CRC-32)	100%	**DONE**	
MotorControlService gRPC	90%	**PARTIAL**	E-Stop priority queue
TelemetryService gRPC	90%	**PARTIAL**	GetSystemStatus stub
BatteryManagementService gRPC	100%	**DONE**	
ConfigurationService gRPC	85%	**PARTIAL**	RetransmitConfig dispatch
FirmwareUpdateService gRPC	0%	**MISSING**	Future enhancement
RESET_ACK auto-response	95%	**PR #349**	PR open
Virtual RX72N simulator	95%	**DONE**	No motor dynamics
star-ros2/			
star_gateway_bridge_node	100%	**DONE**	
star_spi_bridge_node	85%	**PARTIAL**	Never run on hardware
star_safety_monitor	40%	**PARTIAL**	3 of 10 tests skipped
star_bringup launch files	5%	**MISSING**	Empty skeleton
RPLiDAR C1 ROS2 driver	0%	**MISSING**	Host-side sensor
SLAM stack (RTAB-Map, EKF)	0%	**MISSING**	Requires RPLiDAR + IMU
star-ui/			
App scaffold + 7 pages	80%	**PR #351**	Mock data only
Real gRPC / HTTP connection	0%	**MISSING**	Mock client only
Emergency Stop button	0%	**MISSING**	Safety critical
Tests (unit / E2E)	0%	**MISSING**	None yet
star-proto/			
Proto schemas (72 messages)	100%	**DONE**	

Component	Complete	Status	Note / PR
Generated code (Go/TS/-nanopb/C++)	100%	**DONE**	
nanopb motor_status max_count	85%	**PARTIAL**	Bug: 2 should be 4
CI/CD (.github/workflows/)			
proto.yml	100%	**DONE**	
gateway.yml	100%	**DONE**	
ros2.yml	100%	**DONE**	
ui.yml (npm build)	0%	**MISSING**	Not yet created
docs.yml (LaTeX compile)	0%	**MISSING**	Not yet created

Active Pull Requests Summary

PR	Title	What It Fixes	Status
#341	Frame decoder resync	Bounded sync-word recovery	Tests passing
#342	USB→SPI telemetry failover	Telemetry was USB-only in prod	Tests passing
#343	ThreadX stack checking	Stack overflow detection + UART log	Tests passing
#344	BMS SoC thresholds	Warning 25%, critical 5% + config	Tests passing
#345	Remove event-flag infra	Dead code removal (zero callers)	Tests passing
#346	ASCII source cleanup	Replace non-ASCII in 254 files + Doxyfile	Tests passing
#349	Gateway RESET_ACK	Missing case in dispatchControlFrame()	Tests passing
#350	Gap analysis docs	This document	Open
#351	UI dashboard	7 pages with mock data (shadcn/Tailwind)	Manual test

Directory: `e2-studio-star-rx72n-firmware/`

Language: C23 (GNURX toolchain) + ThreadX RTOS
Build: CMake (portable) + Renesas e^2 Studio (IDE)
Hardware: Renesas RX72N (240 MHz, 4 MB Flash, 512 KB SRAM)
Overall: Production-ready. All 9 tasks, all drivers, communication stack complete.

What Is Complete

ThreadX Tasks (9 of 9 implemented)

Task File	Purpose	Lines	Rate
`src/tasks/app_main_task.c`	RTOS entry, task creation	538	startup
`src/tasks/comm_task.c`	SPI/USB frame dispatch	2060	IRQ-driven
`src/tasks/motor_control_task.c`	4-motor PID loop	2870	100 Hz
`src/tasks/telemetry_task.c`	Sensor aggregation + proto	1870	20 Hz
`src/tasks/bms_monitor_task.c`	Battery fault detection	1214	10 Hz
`src/tasks/obstacle_detect_task.c`	HC-SR04 sensor fusion	2202	50 Hz
`src/tasks/temp_sensor_task.c`	DS18B20 1-Wire polling	1227	1 Hz
`src/tasks/led_status_task.c`	Status LED patterns	411	20 Hz
`src/tasks/watchdog_monitor_task.c`	IWDT refresh + stack check	442	100 Hz

Communication Stack

Library	Lines	Capability
`libs/rx_spi_comm/`	2570	RSPI2 @ 10 Mbps, zero-copy buffers
`libs/rx_usb_comm/`	1575	USB CDC-ACM, 3 virtual COM ports
`libs/rx_comm_manager/`	2056	Multi-transport coordinator
`libs/rx_fec/`	869	Convolutional FEC, K=7, rate 1/2
`libs/rx_harq/`	675	Chase combining, configurable retries
`libs/rx_frame/`	895	SYNC+SEQ+LEN+TYPE+FLAGS+CRC-32
`libs/rx_crc/`	2923	CRC-32 (hardware) + CRC-8
`libs/rx_session/`	544	Cross-transport sequence continuity

Hardware Drivers

Driver	Lines	Hardware
`libs/rx_motor/`	1936	DRV8243S H-bridge, PWM dead-time insertion
`libs/rx_pid/`	1582	Discrete-time PID with anti-windup
`libs/rx_encoder/`	1969	MTU1/2 + TPU quadrature encoder channels
`libs/rx_drv8243/`	1983	DRV8243S SPI configuration driver
`libs/rx_bq4050/`	1293	BQ4050 BMS via SMBus (RIIC1)
`libs/rx_ds18b20/`	1429	DS18B20 temperature via 1-Wire
`libs/rx_hcsr04/`	4761	HC-SR04 ultrasonic, IRQ-based echo
`libs/rx_hal/`	16043	Full RX72N HAL (GPIO/SPI/UART/ADC/PWM)
`libs/rx_bus/`	11941	Bus manager (I2C/SPI/1-Wire/UART/SMBus)

WireMessage Command Dispatcher – Confirmed Complete

`comm_task.c:1959` – `internal_handle_command_frame()` handles all incoming commands.

```
/* SetVelocityRequest -> shared_data_set_motor_command() */
/* EmergencyStopRequest -> shared_data_trigger_estop()  */
/* SetPIDGainsRequest -> shared_data_set_pid_gains()    */
/* SetRetransmitConfigRequest -> rx_comm_manager_set_auto_retransmit() */
```

Listing 1: Command dispatcher – handles 4 message types

Active Firmware PRs

PR #341 – Frame decoder resync: bounded linear scan (`k_frame_max_scan_bytes=1036`), new public `rx_frame_decode_with_resync()` API, 3 new Unity tests.

PR #342 – Telemetry transport failover: telemetry was hardcoded to USB only (`k_comm_channel_usb`). Adds `internal_select_transport()` with USB-first, SPI-fallback. Critical production fix.

PR #343 – ThreadX stack checking: enables `TX_ENABLE_STACK_CHECKING`, adds `rx_stack_monitor` module, overflow handler logs thread name then halts (IWDT resets in 2 s). 7 Unity tests.

PR #344 – BMS SoC thresholds: adds `bms_monitor_config_t` with configurable `soc_warning_pct` (25%) and `soc_critical_pct` (5%). 14 new unit tests.

PR #345 – Event-flag cleanup: removes dead `TX_EVENT_FLAGS_GROUP` infrastructure from `shared_data` (`shared_data_wait_event()` had zero callers). All 48 tests pass.

PR #346 – ASCII source cleanup: replaces all non-ASCII charac-

ters across 254 files. Adds `Doxyfile` and Makefile targets (`doxygen`, `doxygen-pdf`).

What Is Missing

Gap F-1: NVS Configuration Persistence (HIGH)

Impact: PID gains received via `SetPIDGainsRequest` are applied in RAM via `shared_data_set_pid_gains()` but NOT saved to flash. Every reboot resets to compile-time defaults – any tuning session must be repeated.
Hardware: RX72N has 64 KB Data Flash (separate from code flash), ideal for NVS.

```
e2-studio-star-rx72n-firmware/
  libs/rx_nvs/
    inc/rx_nvs.h    -- Key-value store API
    src/rx_nvs.c    -- Data Flash implementation
  tests/
    test_rx_nvs.c   -- Unity tests (mock flash)
```

Listing 2: Files to create for NVS

```
/** @brief Write named config blob to Data Flash (max 4096 bytes). */
rx_err_t rx_nvs_write(const char* key,
                      const void* data, uint16_t len);
/** @brief Read named config blob from Data Flash. */
rx_err_t rx_nvs_read(const char* key,
                     void* data, uint16_t* len);
/** @brief Factory reset: erase all Data Flash. */
rx_err_t rx_nvs_erase_all(void);
```

Listing 3: NVS API (rx_nvs.h)

Integration points:

```
// On PID update (comm_task.c):
shared_data_set_pid_gains(...);
rx_nvs_write("pid", &gains, sizeof(gains));

// On boot (main.c, before tx_kernel_enter):
rx_nvs_read("pid", &gains, &len);
shared_data_set_pid_gains(&gains);
```

Estimated effort: 4–6 hours.

Gap F-2: OTA Firmware Update Handler (LOW – Future Enhancement)

Scope note: OTA is not required for the current prototype phase. Hardware can be reflashed over JTAG/SWD during development. This is a future enhancement for production deployment.
Gateway: All 10 `FirmwareUpdateService` RPCs return

`codes.Unimplemented`.
Effort when needed: 2–3 days (firmware state machine + gateway implementation).

Directory: `star-gateway/`

Language: Go 1.22+
Build: `go build ./cmd/star-gateway`
Role: Bridges gRPC clients (UI, ROS2) to RX72N via SPI/USB CDC
Test coverage: 80.8% – enforced in CI via `gateway.yml`
Overall: Transport, framing, FEC, HARQ production-ready. FirmwareUpdateService is a future enhancement.

Directory Structure

```
star-gateway/
  cmd/
    star-gateway/main.go          -- Entry point
    virtual_rx72n/                -- Hardware simulator (95% complete)
  internal/
    transport/                    -- Layer 1: SPI + USB CDC transports
    frame/                        -- Layer 2: SYNC+SEQ+LEN+TYPE+FLAGS+CRC
   -32
    fec/                          -- FEC: Viterbi K=7, rate 1/2
    harq/                         -- HARQ: Chase Combining retransmit
    service/
      motor_control.go            -- MotorControlService (90%)
      telemetry.go                -- TelemetryService (90%)
      battery.go                  -- BatteryManagementService (100%)
      configuration.go            -- ConfigurationService (85%)
      firmware.go                 -- FirmwareUpdateService (0% -- future)
      gateway_service.go          -- GatewayService for ROS2 bridge (85%)
    manager/                      -- Transport manager, RESET_ACK dispatch
    link/                         -- SPI link layer
```

Listing 4: star-gateway/ layout

Test Coverage by Package

Package	Coverage	Status
`internal/controller`	100%	**DONE**
`internal/fec`	100%	**DONE**
`internal/frame`	100%	**DONE**
`internal/transport`	92.1%	**DONE**
`internal/manager`	97.4%	**DONE**
`internal/service`	94.4%	**PARTIAL** (firmware.go is future)
`internal/harq`	87.0%	**DONE**
`internal/link`	83.3%	**DONE**

Virtual RX72N Simulator (`cmd/virtual_rx72n/`)

Feature	Status
Full SPI frame protocol	**DONE**
PING/PONG heartbeat with counter echo	**DONE**
RESET/RESET_ACK session sync	**DONE**
VelocityCommand + EmergencyStop handling	**DONE**
Simulated TelemetryData (static values)	**DONE**
Sequence number tracking with wraparound	**DONE**
ACK frame generation (FlagRequiresAck)	**DONE**
Configurable latency (SIM_LATENCY_MS env var)	**DONE**
Graceful shutdown with socket cleanup	**DONE**
Motor dynamics simulation G(s)=3.665/(0.075s+1)	**MISSING**
Fault injection API (CRC errors, packet loss)	**MISSING**
BMS state-of-charge discharge simulation	**MISSING**

To enable integration testing without hardware, enhance the simulator with motor dynamics:

```
// First-order: G(s) = K/(tau*s + 1),  K=3.665 rad/s/V, tau=0.075 s
// Discrete backward Euler at T=0.010 s:
//   v[k] = 0.1248*K*u[k] + 0.8752*v[k-1]
alpha := math.Exp(-motorSamplePeriod / motorTimeConstant)
m.velocityRadps = (1-alpha)*motorGain*inputVoltage + alpha*m.velocityRadps
m.positionRad  += m.velocityRadps * motorSamplePeriod
ticks          := int64(m.positionRad / (2*math.Pi) * ticksPerRev)
m.encoderTicks.Store(ticks)
```

Listing 5: Motor dynamics model for virtual_rx72n/motor_sim.go

What Is Missing

Gap G-1: RESET_ACK Auto-Response (IN PROGRESS)

PR #349 – `dispatchControlFrame()` was missing `case frame.FrameTypeReset:`, causing firmware-initiated RESET frames to be silently discarded. Adds `sendResetAck()` modelled after `sendPong()` with operations-gate bypass to avoid deadlock. All 13 packages pass.

Gap G-2: E-Stop Priority Queue (MEDIUM)

File: `internal/service/motor_control.go` – TODO comment: "Implement priority framing for E-Stop."
Impact: During link congestion, E-Stop may be delayed behind pending ACKs.
Required: Bypass the normal HARQ queue for emergency stop frames.
Estimated effort: 2–3 hours.

Gap G-3: SetMotorPower Not Dispatching (MEDIUM)

`SetMotorPower` RPC exists but does not send `MotorPowerCommand` to the RX72N. The firmware dispatcher also has no handler for it. Requires a coordinated change to `wire.proto`, firmware dispatcher, and gateway service.

Estimated effort: 3–4 hours (proto + firmware + gateway).

Gap G-4: Virtual RX72N Motor Dynamics (LOW)

Simulator returns static telemetry regardless of velocity commands. Adding motor dynamics (see code above) would enable odometry and PID testing without hardware.

Estimated effort: 1–2 days.

Directory: `star-ros2/`

Language: C++17, ROS2 Jazzy
Build: `colcon build` + `clang-format` enforcement
Role: Middleware – bridges gateway gRPC to robot topics, manages safety, hosts RPLiDAR
Overall: Core bridge nodes complete. Hardware integration untested. SLAM stack not started.

Package Structure

```
star-ros2/
  star_gateway_bridge/       -- gRPC client <-> ROS2 topics (COMPLETE)
  star_spi_bridge/           -- SPI driver ROS2 node (code complete,
   untested)
  star_safety_monitor/       -- Safety watchdog (40% test coverage)
  star_bringup/              -- Launch files (EMPTY SKELETON)
  star_nav/                  -- Navigation stack (0%)
```

Listing 6: star-ros2/ package layout

What Is Complete

`star_gateway_bridge` (100%)

Fully implemented. Bridges gateway gRPC service to ROS2 topics.

Function	ROS2 Interface	gRPC Call
Teleop forwarding	`/velocity_command` (sub)	`GetTeleopCommand()`
Telemetry bridge	`/robot_status` (pub)	`ForwardTelemetry()`
PID gain update	`/pid_gains` (sub)	`SetPIDGains()`

`star_spi_bridge` (85% – code complete, untested)

All source code is written. This package has **never been run on real hardware** (RPi5 + RX72N). This is the final integration point before the robot can physically move.

Known safety gap in `src/star_spi_driver_node.cpp`:

```
// CURRENT (unsafe): closes SPI while motors may still be spinning
CallbackReturn on_deactivate(const State&) override {
    transport_->close();
    return CallbackReturn::SUCCESS;
}

// REQUIRED (safe):
CallbackReturn on_deactivate(const State&) override {
    auto zero = build_velocity_command(0.0f, 0.0f, 0.0f, 0.0f);
```

```
    transport_->send(zero);
    std::this_thread::sleep_for(std::chrono::milliseconds(50));
    transport_->close();
    return CallbackReturn::SUCCESS;
}
```

Listing 7: on_deactivate() must send zero velocity before closing SPI

What Is Missing

Gap R-1: SPI Bridge Hardware Testing (HIGH)

Effort: 4–8 hours on RPi5 + RX72N hardware.
Validates: Frame encoding, HARQ retransmit, telemetry reception, encoder feedback, 100 Hz polling requirement (must not miss cycles to avoid 500 ms RX72N timeout).

Pre-test checklist:

- Fix `on_deactivate()` zero-velocity safety (above)
- Verify SPI CS polarity (active-low) matches firmware
- Confirm SPI Mode 0 @ 10 MHz in RPi5 device tree
- Verify COPI/CIPO pin assignments against hardware pinout

Gap R-2: `star_bringup` Launch Files (HIGH)

`star_bringup/launch/` is completely empty. Cannot start the system without manually launching each node.

```
star_bringup/launch/
  star_robot.launch.py        -- Full system (bridge + SPI + safety + nav)
  star_simulation.launch.py   -- Simulator (Gazebo + virtual_rx72n)
  star_teleop.launch.py       -- Manual control only
  star_nav.launch.py          -- Autonomous (adds SLAM + NavStack)
```

Listing 8: Required launch files

Estimated effort: 4–6 hours.

Gap R-3: Safety Monitor Test Coverage (MEDIUM)

`star_safety_monitor/` – 3 of 10 test cases use `GTEST_SKIP`.
Battery critically low, over-temperature, and communication timeout paths are not tested.
Estimated effort: 4–6 hours.

Gap R-4: RPLiDAR C1 ROS2 Integration (MEDIUM)

Location note: The RPLiDAR C1 sensor connects to the **Raspberry Pi**

5 (not the RX72N). It runs as a ROS2 node on the host, not as firmware. Use the existing `rplidar_ros2` community package.

Item	What Is Needed
ROS2 package	Add `rplidar_ros2` to workspace dependencies
Launch integration	Include lidar node in `star_bringup` launch files
Topic remapping	`/scan` topic available for SLAM consumption
Hardware mount	Physical UART/USB connection from RPi5 to RPLiDAR

Estimated effort: 4–8 hours (driver setup + launch integration + SLAM config).

Gap R-5: SLAM Stack (LOW – requires RPLiDAR + IMU first)

Component	Status
RTAB-Map (mapping + localization)	Not installed or configured
`robot_localization` EKF	Not configured
Nav2 (path planning)	Not installed or configured
IMU (MPU-6050 or ICM-42688)	Hardware not yet chosen – connects to RPi5

Prerequisite: RPLiDAR C1 ROS2 integration (Gap R-4) must work first.

Directory: `star-ui/`

Language: TypeScript, React 19, Tailwind v4, shadcn/ui
Build: `npm run build` (Vite)
Overall: All 7 pages scaffolded with mock data in PR #351. Real data connection is the next step.

In Progress – PR #351

PR #351 – Complete STAR dashboard frontend. Adds 7 pages with dark-themed shadcn/ui components and a `MockGatewayClient` that returns realistic simulated data. The UI is fully navigable and interactive without hardware.

Pages added: Dashboard, Controller, Telemetry, Battery, Motors, Configuration, Firmware

Architecture: `GatewayClient` interface + `MockGatewayClient` implementation. Switching to real data requires implementing the real `GatewayClient` class.

Status: Manual test plan only (no automated tests yet).

What Is Missing

Gap U-1: Real Gateway Connection (HIGH – replaces MockGatewayClient)

Impact: The UI is fully built (PR #351) but uses mock data. Connecting to the real gateway requires a gRPC-Web or REST bridge between the browser and Go gRPC server.

Options:

1. **grpc-gateway** (recommended, 4 hrs): REST/JSON transcoding to existing gRPC services. No extra process.
2. **Envoy proxy** (production-grade): HTTP/2 proxy as a sidecar on RPi5. More complex.
3. **connect-go** (modern): Replace grpc-go with connect-go for native HTTP/1.1.

```
mux := runtime.NewServeMux()
opts := []grpc.DialOption{grpc.WithTransportCredentials(
    insecure.NewCredentials())}
// Register all 5 services for REST transcoding:
starv1.RegisterMotorControlServiceHandlerFromEndpoint(
    ctx, mux, "localhost:50051", opts)
starv1.RegisterTelemetryServiceHandlerFromEndpoint(
    ctx, mux, "localhost:50051", opts)
// ... repeat for Battery, Configuration, Firmware
http.ListenAndServe(":8080", mux)
```

Listing 9: grpc-gateway setup in cmd/star-gateway/main.go

Estimated effort: 4–8 hours (bridge setup + real GatewayClient class in UI).

Gap U-2: Emergency Stop Button (HIGH – safety critical)

No E-Stop button exists in the UI. Operators can only stop via gamepad button – no visible on-screen control is present. This is a safety gap.

Required: Prominent red "EMERGENCY STOP" button always visible in the header. Keyboard shortcut (Space or Escape). Calls the E-Stop RPC. Shows ACTIVE/CLEARED state.

Estimated effort: 2–3 hours (after real gateway connection working).

Gap U-3: Tests (MEDIUM)

Currently 0% test coverage. Vitest is configured.

Recommended:

- Vitest + React Testing Library for unit tests (component behaviour)
- Playwright for E2E (full page interaction)

Estimated effort: 8–12 hours for basic coverage.

Directory: `star-proto/`

Language: Protocol Buffers 3 (buf toolchain)
Targets: Go, TypeScript, nanopb C, C++ (gRPC)
Overall: Schemas are complete. One known nanopb options bug. All generated code is checked in.

Schema Inventory

Proto File	Msgs	Services	Status
`common.proto`	5	–	**DONE**
`motor_control.proto`	9	1 (5 RPC)	**DONE**
`telemetry.proto`	7	1 (3 RPC)	**DONE**
`battery_management.proto`	15	1 (10 RPC)	**DONE**
`configuration.proto`	13	1 (7 RPC)	**DONE**
`firmware_update.proto`	12	1 (10 RPC)	**DONE** (future use)
`gateway_service.proto`	6	1 (3 RPC)	**DONE**
`diagnostics.proto`	2	–	**DONE**
`controller.proto`	1	–	**DONE**
`wire.proto`	2	–	**DONE**
Total	**72**	**5 svcs / 38 RPC**	

Known Bug: nanopb motor_status Truncation

File: `nanopb/star/v1/motor_control.options`
Bug: `star.v1.SetVelocityResponse.motor_status max_count:2`
Fix: Change to `max_count:4` (the robot has 4 motors: FL, FR, BL, BR)
Impact: Response messages silently truncate motors 3 and 4.
Fix effort: 30 minutes – edit file, run `make proto-gen && make proto-gen-firmware`.

Missing Wire Protocol Messages (for OTA – future)

When OTA is implemented, `wire.proto` will need new types added to the `WireMessage` oneof (not required for the current prototype):

- `FirmwareChunk` – binary chunk payload
- `BeginUpdateRequest` – OTA session initialiser
- `FinalizeUpdateRequest` – CRC validation trigger

Directory: `.github/workflows/`

Existing CI: proto.yml, gateway.yml, ros2.yml (all working)
Missing CI: UI build, docs compile
Note: Firmware CI is **not planned** – the GNURX cross-compiler toolchain is too large to install in CI runners.

Existing Workflows

Workflow	Triggers	Steps
`proto.yml`	Push to `star-proto/`	buf lint, breaking check, generate, Go/TS/nanopb tests
`gateway.yml`	Push to `star-gateway/`	build, test (-race), lint, security scan, cross-compile (amd64/arm64)
`ros2.yml`	Push to `star-ros2/`	colcon build, ament_lint, code review
`proto-gen.yml`	Push to `*.proto`	buf format, generate, check for uncommitted changes

What Is Missing

Gap C-1: UI Build CI (`ui.yml`) – MEDIUM

No CI exists for `star-ui/`. TypeScript errors and broken builds can merge undetected.

```
steps:
  - npm ci
  - npm run type-check   # TypeScript strict mode
  - npm run lint         # ESLint
  - npm run test         # Vitest unit tests
  - npm run build        # Production Vite bundle (catches issues)
```

Listing 10: Required steps for ui.yml

Estimated effort: 2–3 hours.

Gap C-2: Documentation Build CI (`docs.yml`) – LOW

```
steps:
  - apt-get install texlive-xetex texlive-fonts-extra
  - cd docs && make        # Compile star_documentation.pdf
  - Upload PDF as artifact
```

Listing 11: Required steps for docs.yml

Estimated effort: 1–2 hours.

Implementation Roadmap

Phase 0: Merge Open PRs (in flight)

All 7 open firmware/infra PRs are ready (tests passing). Merge in order to avoid conflicts:

1. **PR #345** – Remove event-flag infra (no conflicts, merge first)
2. **PR #342** – Telemetry USB/SPI failover (critical production fix)
3. **PR #341** – Frame decoder resync
4. **PR #343** – ThreadX stack checking
5. **PR #344** – BMS SoC thresholds (depends on #345 merged first)
6. **PR #346** – ASCII source cleanup (touches many files, merge last)
7. **PR #349** – Gateway RESET_ACK fix
8. **PR #351** – UI dashboard (review mock architecture)

Phase 1: Quick Wins (1–2 hours)

1. **Fix nanopb motor_status bug** (`star-proto/`): change `max_count:2` to `max_count:4`
2. **Fix SPI bridge zero-velocity safety** (`star-ros2/star_spi_bridge/`): send zero-vel in `on_deactivate()`

Phase 2: First Robot Motion (2–4 days on hardware)

3. **Hardware bring-up:** verify SPI/COPI/CIPO/CS wiring matches firmware pinout
4. **Run SPI bridge on hardware:** test with virtual_rx72n first, then real firmware
5. **Create star_bringup launch files:** minimum `star_robot.launch.py`
6. **Validate teleop path:** gamepad → gateway → ROS2 → SPI bridge → motors spin

Phase 3: Real UI Data (1–2 weeks)

7. **Implement real GatewayClient** in `star-ui/` (replaces MockGatewayClient)
8. **Add grpc-gateway** or REST bridge to gateway (browser access to gRPC services)
9. **Emergency Stop button** (safety-critical UI component)
10. **UI CI** (`ui.yml` workflow)

Phase 4: Robustness (1–2 weeks)

11. **NVS library** (`libs/rx_nvs/`): PID gain persistence across reboot
12. **E-Stop priority queue** in gateway: bypass normal HARQ queue
13. **Safety monitor tests**: implement 3 skipped test cases
14. **Virtual RX72N motor dynamics**: enables sim-based PID tuning

Phase 5: SLAM and Autonomy (future)

15. RPLiDAR C1 ROS2 integration on RPi5 (`rplidar_ros2` package)
16. RTAB-Map configuration for ROS2 Jazzy
17. `robot_localization` EKF with encoder odometry
18. Nav2 path planner

Effort Summary

Gap	Severity	Effort	Status
Fix nanopb max_count:4	Critical bug	30 min	Not started
SPI on_deactivate() zero-vel	Safety	1 hr	Not started
Merge open PRs (#341–#351)	High	–	**PR #341-349**
star_bringup launch files	High	4–6 hrs	Not started
SPI bridge hardware testing	High	4–8 hrs	Not started
Real GatewayClient (UI)	High	4–8 hrs	**PR #351** (mock)
gRPC-Web / REST bridge	High	4–8 hrs	Not started
Emergency Stop UI button	High (safety)	2–3 hrs	Not started
Safety monitor test coverage	Medium	4–6 hrs	Not started
NVS library (rx_nvs)	Medium	4–6 hrs	Not started
PID gain persistence	Medium	2 hrs	Needs NVS first
E-Stop priority queue	Medium	2–3 hrs	Not started
UI CI (ui.yml)	Medium	2–3 hrs	Not started
SetMotorPower dispatch	Medium	3–4 hrs	Not started
Docs CI (docs.yml)	Low	1–2 hrs	Not started
Virtual RX72N motor dynamics	Low	1–2 days	Not started
Virtual RX72N fault injection	Low	4–8 hrs	Not started
RPLiDAR C1 ROS2 (on RPi5)	Low	4–8 hrs	Not started
OTA firmware update	Future	2–3 days	Not required (prototype)
SLAM stack	Future	weeks	Not started